

Ncc Sign on ARMv8

유튜브 주소 : <https://youtu.be/Lxy0Tow-upY>

NCC Sign

- KpqC공모전에 제출된 전자서명 알고리즘
 - Round1을 통과하여 Round 2에 진출, 최종 선정 X
- **격자 기반으로 개발**
 - RLWE (Ring Learning With Errors)
 - 고차원의 격자 문제(LWE)를 다항식 구조로 확장한 형태
 - 비순환 다항식(Non-Cyclotomic Polynomial)을 사용하여 RLWE 문제를 설정
 - 기존 순환 다항식 구조의 대수적 약점을 제거
 - RSIS (Ring Short Integer Solution)
 - 기존의 Short Integer Solution (SIS) 문제를 다항식 링 구조로 확장한 형태
 - 서명 생성 시 RSIS 난제 활용
 - SelfTargetRSIS
 - RLWE와 RSIS를 조합한 형태로, 서명 검증 시 활용
- 다항식 및 NTT(수 이론 변환)를 활용하여 설계
- **NTT가 주요 연산**

Montgomery reduction 최적 구현 코드

- 4계수씩 벡터 레지스터에 로드하여 병렬 연산
- 핵심 명령어
 - Sqdmulh(vqdmulhq_s32): 상위 32비트 곱셈
 - Mul(vmulq_s32): 32비트 정수 곱셈, 하위 32비트 저장
 - 모듈러 연산을 해야하므로 상위 비트 필요 X
 - Shsub(vqdmulhq_s32): 비트 간 뺄셈 후 $\frac{1}{2}$
- C에서 루프 4번 반복해야 할 연산을 한 번에 처리
- Asm -> intrinsic으로 구현
 - 레퍼런스 C 구현 대비 6.8배 정도 향상
 - Asm 구현 대비 2.3배 정도 향상

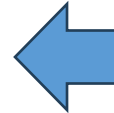
```
////////// ref //////////  
int32_t montgomery_reduce(int64_t a) {  
    int32_t t;  
  
    t = (int64_t)(int32_t)a*QINV;  
    t = (a - (int64_t)t*Q) >> 32;  
    return t;  
}
```



```
montgomery_reduce:  
_montgomery_reduce:  
    // [1] Q 값 설정 (v3에 Q 값 저장)  
    // 0xfc = 252; rev16 처리하여 0xfc00; 또는 직접 상수를  
    // [3] 입력 a, b 로드  
    ld1    {v0.4s}, [x1]           // v0: 4개의 int32_t 값 from a  
    ld1    {v2.4s}, [x2]           // v1: 4개의 int32_t 값 from b  
    ld1R   {v3.4s}, [x3], #4       // v1: 4개의 int32_t 값 from b  
    ld1R   {v4.4s}, [x3]  
  
    sqdmulh v6.4s, v0.4s, v2.4s  
    mul.4s v27, v2, v4  
    mul.4s v7, v0, v27  
    sqdmulh v16.4s, v7.4s, v3.4s  
    shsub.4s v6, v6, v16  
  
    // 결과를 out 배열에 저장  
    st1    {v6.4s}, [x0]  
  
    ret
```



```
int32x4_t montgomery_mul4_sqdmulh(int32x4_t b,  
                                   int32x4_t vZ,  
                                   int32x4_t vQ,  
                                   int32x4_t vQINV)  
{  
    int32x4_t high_ab = vqdmulhq_s32(vZ, b); // high(2*z*b)  
    int32x4_t b_qinv  = vmulq_s32(b, vQINV); // b*QINV (low32)  
    int32x4_t a_times = vmulq_s32(vZ, b_qinv); // z*(b*QINV)  
    int32x4_t high_corr = vqdmulhq_s32(a_times, vQ); // high(2*(...)*Q)  
    return vhsbq_s32(high_ab, high_corr); // (high_ab - high_corr)/2  
}
```



```
[Performance Test - 1000000 repetitions]  
Ref  time: 0.002987 sec (per 4 results)  
ASM  time: 0.001030 sec  
NEON time: 0.000435 sec  
Speedup ASM vs Ref : 2.90x  
Speedup NEON vs Ref : 6.87x  
Speedup NEON vs ASM : 2.37x
```

NCC-Sign3 NTT 구현(neon intrinsic)

- Reduce32_vec
 - Lazy modular reduction 수행 함수
 - 23-bit 쉬프트로 $x - t*Q$ 계산
- Stage0_neon
 - NTT 첫 라운드 버터플라이 연산
 - $T = z*b$ 4배수 계산

```
static inline __attribute__((always_inline))
int32x4_t reduce32_vec(int32x4_t x) {
    const int32x4_t add = vdupq_n_s32(1 << 22);
    const int32x4_t qv = vdupq_n_s32(Q);
    int32x4_t t = vaddq_s32(x, add);
    t = vshrq_n_s32(t, 23); // t = (x + 2^22) >> 23
    int32x4_t tq = vmulq_s32(t, qv); // low32(t * Q)
    return vsubq_s32(x, tq); // x - t*Q
}
```

```
static inline void stage0_neon(int32_t *restrict Out,
                               const int32_t *restrict zetas,
                               int32x4_t vQ,
                               int32x4_t vQINV)
{
    const int z = zetas[0];
    const int n2 = N >> 1; // 768
    const int32x4_t vZ = vdupq_n_s32(z);

    for (int j = 0; j < n2; j += 4) {
        int32x4_t a = vld1q_s32(&Out[j]);
        int32x4_t b = vld1q_s32(&Out[j + n2]);
        int32x4_t t = montgomery_mul4_sqdmulh(b, vZ, vQ, vQINV);
        int32x4_t a_plus_t = vaddq_s32(a, t);
        int32x4_t a_plus_b = vaddq_s32(a, b);
        int32x4_t b_new = vsubq_s32(a_plus_b, t); // a + b - t
        vst1q_s32(&Out[j], a_plus_t);
        vst1q_s32(&Out[j + n2], b_new);
    }
}
```

NCC-Sign3 NTT 구현(neon intrinsic)

- Radix2_upper_neon
 - $\text{Len} = 4/N \sim \text{radix2_redlen_ntt}$ 까지의 버터플라이 연산 처리
- Radix2_lower_neon
 - $\text{Len} = \text{radix2_redlen_ntt} \sim 3 * \text{radix3_len}$ 까지의 버터플라이 연산 처리

```
static inline void radix2_upper_neon(int32_t *restrict Out,
                                     const int32_t *restrict zetas,
                                     int *pk,
                                     int32x4_t vQ,
                                     int32x4_t vQINV)
{
    for (int len = N >> 2; len > RAD2_REDLLEN_C; len >>= 1) {
        for (int start = 0; start < N; start += (len << 1)) {
            const int z = zetas[(*pk)++];
            const int32x4_t vZ = vdupq_n_s32(z);

            int j = start, j_end = start + len;
            for (; j + 4 <= j_end; j += 4) {
                int32x4_t a = vld1q_s32(&Out[j]);
                int32x4_t b = vld1q_s32(&Out[j + len]);
                int32x4_t t = montgomery_mul4_sqdmulh(b, vZ, vQ, vQINV);
                vst1q_s32(&Out[j + len], vsubq_s32(a, t)); // b' = a - t
                vst1q_s32(&Out[j], vaddq_s32(a, t)); // a' = a + t
            }
            // 스칼라 테일(최대 3)
            for (; j < j_end; ++j) {
                int32_t a = Out[j];
                int32_t b = Out[j + len];
                int32_t t = montgomery_reduce((int64_t)z * (int64_t)b);
                Out[j + len] = a - t;
                Out[j] = a + t;
            }
        }
    }
}
```

```
static inline void radix2_lower_neon(int32_t *restrict Out,
                                     const int32_t *restrict zetas,
                                     int *pk,
                                     int32x4_t vQ,
                                     int32x4_t vQINV)
{
    for (int len = RAD2_REDLLEN_C; len >= 3 * RAD3_LEN_C; len >>= 1) {
        for (int start = 0; start < N; start += (len << 1)) {
            const int z = zetas[(*pk)++];
            const int32x4_t vZ = vdupq_n_s32(z);

            int j = start, j_end = start + len;
            for (; j + 4 <= j_end; j += 4) {
                int32x4_t a = vld1q_s32(&Out[j]);
                int32x4_t b = vld1q_s32(&Out[j + len]);
                int32x4_t t = montgomery_mul4_sqdmulh(b, vZ, vQ, vQINV);
                vst1q_s32(&Out[j + len], vsubq_s32(a, t));
                vst1q_s32(&Out[j], vaddq_s32(a, t));
            }
            for (; j < j_end; ++j) {
                int32_t a = Out[j];
                int32_t b = Out[j + len];
                int32_t t = montgomery_reduce((int64_t)z * (int64_t)b);
                Out[j + len] = a - t;
                Out[j] = a + t;
            }
        }
    }
}
```

NCC-Sign3 NTT 구현(neon intrinsic)

- NTT 메인 함수 최적 구현
- NTT 성능 약 2.5~6배 향상

```
void ntt_neon_intrin(int32_t *out,
                    const int32_t *in,
                    const int32_t *zetas)
{
    if (out != in) memcpy(out, in, sizeof(int32_t)*N);

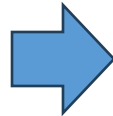
    // vQ/vQINV는 함수 수명 전체에서 재사용(호이스트)
    const int32x4_t vQ    = vdupq_n_s32(Q);
    const int32x4_t vQINV = vdupq_n_s32(QINV);

    stage0_neon(out, zetas, vQ, vQINV);

    int k = 1; // zetas[0] 소비
    radix2_upper_neon(out, zetas, &k, vQ, vQINV);

    // 중간 reduce32 (벡터 1패스)
    for (int i = 0; i < N; i += 4) {
        int32x4_t x = vld1q_s32(&out[i]);
        x = reduce32_vec(x);
        vst1q_s32(&out[i], x);
    }

    radix2_lower_neon(out, zetas, &k, vQ, vQINV);
    // (mode 3은 radix-3 없음)
}
```



```
=== NTT Perf (iters=500, rounds=5) ===
C REF : best 4452.8 ns/op, avg 4647.5 ns/op (chk=00a33b62)
NEON  : best 1720.6 ns/op, avg 1789.8 ns/op (chk=00a33b62)
Speedup: best 2.59x, avg 2.60x
```

```
[C REF] OK
[NEON ] OK
golden (first 16):
1028305, 8246112, 5567146, 2020271, 6128135, 4538838, 2551212, 1185403,
8277771, 6758728, 7615312, 163249, 2192053, 1621392, 4443908, 3408288
neon (first 16):
1028305, 8246112, 5567146, 2020271, 6128135, 4538838, 2551212, 1185403,
8277771, 6758728, 7615312, 163249, 2192053, 1621392, 4443908, 3408288
```

NCC-Sign InvNTT 구현

```
void invntt_neon_intrin(int32_t *restrict out,
                        const int32_t *restrict in,
                        const int32_t *restrict zetas_inv)
{
    #if (NIMS_TRI_NTT_MODE == 3)

        if (out != in) memcpy(out, in, sizeof(int32_t)*N);

        extern void invntt_ref_mode3_compat(int32_t*, const int32_t*, const int32_t*);
        invntt_ref_mode3_compat(out, in, zetas_inv);
        return;
    #else
        if (out != in) memcpy(out, in, sizeof(int32_t)*N);

        const int32x4_t vQ = vdupq_n_s32(Q);
        const int32x4_t vQINV = vdupq_n_s32(QINV);

        int k = 0;

        // (1) radix-3 (상항) - C 레퍼런스와 동일한 순서
        radix3_inverse_mixed_neon(out, zetas_inv, &k, vQ, vQINV);

        // (2) radix-2 하단: len = 3*radix3_len; len < radix2_redlen_invntt; len <= 1
        for (int len = 3 * radix3_len; len < radix2_redlen_invntt; len <= 1) {
            for (int start = 0; start < N; start += (len << 1)) {
                int32x4_t vZinv = vdupq_n_s32(zetas_inv[k++]);

                int j = start, j_end = start + len;
                for (; j + 4 <= j_end; j += 4)
                    inv_bfly4_store(out + j, out + j + len, vZinv, vQ, vQINV);
            }
        }
    #endif
}
```

```
// (3) 중간 reduce32
for (int i = 0; i < N; i += 4) {
    int32x4_t x = vld1q_s32(out + i);
    x = reduce32_vec(x);
    vst1q_s32(out + i, x);
}

// (4) radix-2 상단: len = radix2_redlen_invntt; len <= N/4; len <= 1
for (int len = radix2_redlen_invntt; len <= (N >> 2); len <= 1) {
    for (int start = 0; start < N; start += (len << 1)) {
        int32x4_t vZinv = vdupq_n_s32(zetas_inv[k++]);

        int j = start, j_end = start + len;
        for (; j + 4 <= j_end; j += 4)
            inv_bfly4_store(out + j, out + j + len, vZinv, vQ, vQINV);

        for (; j < j_end; ++j) {
            int32_t a = out[j], b = out[j + len];
            out[j] = a + b;
            out[j + len] = montgomery_reduce((int64_t)zetas_inv[k-1] * (a - b));
        }
    }
}

// (5) 최종 결산: z = zetas_inv[k] (증가 X), F1/F2 스케일
{
    const int n2 = N >> 1;
    int32x4_t vF1 = vdupq_n_s32(F1);
    int32x4_t vF2 = vdupq_n_s32(F2);
    int32x4_t vZ0 = vdupq_n_s32(zetas_inv[k]); // k++ 아님

    for (int j = 0; j < n2; j += 4) {
        int32x4_t a = vld1q_s32(out + j);
        int32x4_t b = vld1q_s32(out + j + n2);
        int32x4_t sum = vaddq_s32(a, b);
        int32x4_t diff = vsubq_s32(a, b);
        int32x4_t t = montgomery_mul4_sqdmulh(diff, vZ0, vQ, vQINV);
        int32x4_t a_new = montgomery_mul4_sqdmulh(vsubq_s32(sum, t), vF1, vQ, vQINV);
        int32x4_t b_new = montgomery_mul4_sqdmulh(t, vF2, vQ, vQINV);
        vst1q_s32(out + j, a_new);
        vst1q_s32(out + j + n2, b_new);
    }
}
#endif
}
```


NTT radix3 파트 구현

```
#if (NIMS_TRI_NTT_MODE != 3)
static inline void radix3_forward_mixed_neon(int32_t *restrict Out,
                                             const int32_t *restrict zetas,
                                             int *pk,
                                             int32x4_t vQ,
                                             int32x4_t vQINV)
{
    const int32x4_t vW = vdupq_n_s32(Wmont);
    const int32x4_t vW2 = vdupq_n_s32(W2mont);

    for (int len = radix3_len; len >= 1; len /= 3) {
        for (int start = 0; start < N; start += 3 * len) {
            const int z1 = zetas[(*pk)++];
            const int z2 = zetas[(*pk)++];
            const int32x4_t vZ1 = vdupq_n_s32(z1);
            const int32x4_t vZ2 = vdupq_n_s32(z2);

            int j = start, j_end = start + len;

            for (; j + 4 <= j_end; j += 4) {
                int32x4_t a0 = vld1q_s32(&Out[j]);
                int32x4_t a1 = vld1q_s32(&Out[j + len]);
                int32x4_t a2 = vld1q_s32(&Out[j + 2*len]);

                int32x4_t t1 = montgomery_mul4_sqdmulh(a1, vZ1, vQ, vQINV);
                int32x4_t t2 = montgomery_mul4_sqdmulh(a2, vZ2, vQ, vQINV);
                int32x4_t t3 = montgomery_mul4_sqdmulh(t1, vW, vQ, vQINV); // w * t1
                int32x4_t t4 = montgomery_mul4_sqdmulh(t2, vW2, vQ, vQINV); // w^2 * t2
            }
        }
    }
}
```

```
int32x4_t t12 = vaddq_s32(t1, t2);
int32x4_t t34 = vaddq_s32(t3, t4);

int32x4_t a2n = vsubq_s32(a0, vaddq_s32(t12, t34));
int32x4_t a1n = vaddq_s32(a0, t34);
int32x4_t a0n = vaddq_s32(a0, t12);

vst1q_s32(&Out[j], a0n);
vst1q_s32(&Out[j + len], a1n);
vst1q_s32(&Out[j + 2*len], a2n);
}
for (; j < j_end; ++j) {
    int32_t a0 = Out[j];
    int32_t a1 = Out[j + len];
    int32_t a2 = Out[j + 2*len];

    int32_t t1 = montgomery_reduce((int64_t)z1 * a1);
    int32_t t2 = montgomery_reduce((int64_t)z2 * a2);
    int32_t t3 = montgomery_reduce((int64_t)Wmont * t1);
    int32_t t4 = montgomery_reduce((int64_t)W2mont * t2);

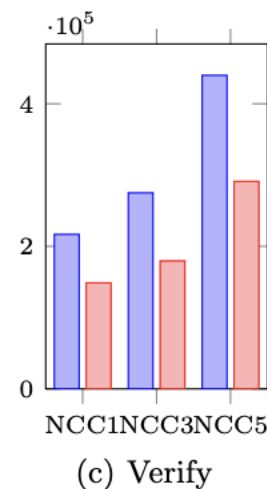
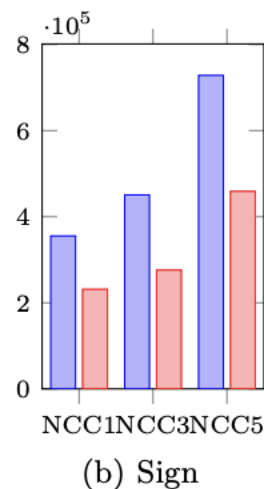
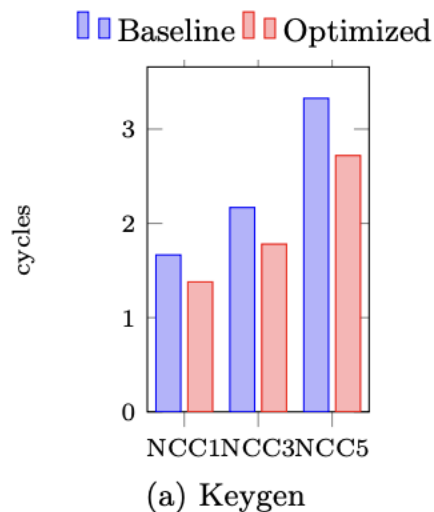
    int32_t t12 = t1 + t2;
    int32_t t34 = t3 + t4;

    Out[j + 2*len] = a0 - (t12 + t34);
    Out[j + len] = a0 + t34;
    Out[j] = a0 + t12;
}
}
#endif
```


NCC Sign 1,3,5 성능 측정

- Keygen: 약 20% 향상
- Sign: 30~40% 향상
- Verify: 30~40% 향상

Param	Keygen		Sign		Verify	
	Baseline	Optimized	Baseline	Optimized	Baseline	Optimized
NCC-Sign-1	166545	137888 (1.21×)	355311	231496 (1.53×)	216822	148735 (1.46×)
NCC-Sign-3	216841	178021 (1.22×)	450648	276123 (1.63×)	275006	179491 (1.53×)
NCC-Sign-5	332706	271995 (1.22×)	728419	459016 (1.59×)	439860	291103 (1.51×)
Geo-mean speedup		1.22×		1.58×		1.50×



Q & A